CHAPTER 31

Proving Programs Correct

Chapter 1 ended its first look at Lean with a comparison. A Rust loop checked Gauss's formula for 10,001 numbers; a Lean theorem proved it for all of them. The chapter promised that we would come back to this division of labor and use it deliberately. This is the chapter where we do.

Chapter 30 gave us types that can say precise things: a vector of length `n`, an index smaller than the length, a value together with a proof about it. Here we push the idea to its natural end. We write ordinary programs (a sort and a binary search), state what they are supposed to do as types, prove that they do it, and run them. Then we write the same programs in Rust, the way a Rust programmer would, and test the Rust code against the verified Lean model. Along the way we need the tools that make such proofs practical: induction with the right hypothesis, termination arguments for functions that are not structurally recursive, and a working knowledge of the tactics that do the routine steps for us.

The chapter ends with a survey of the tools that verify Rust code directly, and with an honest account of what a proof gives you and what it does not.

In this chapter you will learn:

- how to write a specification as a type, and why a good specification usually needs more than one property;
- induction on lists and trees, and when the induction hypothesis must be generalized;
- structural and well-founded recursion: `termination_by`, `decreasing_by`, `partial`, fuel, and proofs by functional induction;
- a verified insertion sort and a verified binary search, stated with Mathlib's `List.SortedLE` and `List.Perm`;
- tactics in more depth: simp lemmas and `simp?`, `omega`, `linarith`, `ring`, `calc` and `grind`;
- the workflow "model in Lean, implement in Rust", with differential and property-based tests;
- which tools verify Rust code itself, and what a proof does and does not guarantee.

31.1 Specifications as types

Here are three ways to say, in Lean, that a function sorts a list of integers.

The first is the type `List ℤ → List ℤ`. It says almost nothing. The identity function has this type, and so do `List.reverse` and `fun _ => []`. A type like this is all that Rust's `fn sort(v: Vec<i64>) -> Vec<i64>` can say.

The second is a function of that type together with separate theorems about it: "for every `xs`, `sort xs` is sorted" and "for every `xs`, `sort xs` is a permutation of `xs`". This is the *extrinsic* style that Chapter 8 contrasted with intrinsic typing: the program is written first, and its properties are proved afterwards, outside its type.

The third puts the properties *in* the type, with a subtype (Chapter 30):

```
def sort (xs : List ℤ) : { ys : List ℤ // ys.SortedLE ∧ ys ~ xs }
```

A value of this type is a list together with a proof that it is sorted and a proof that it is a permutation of the input. A function of this type cannot return a wrong answer, because a wrong answer would come without its proofs. This is the *intrinsic* style, and it is the form in which "a specification is a type" becomes literal. In practice the two styles combine: we write the program and prove the theorems separately, then package them into the subtype at the end, as you will see.

Notice that the specification has two parts, and that neither is enough alone. "Sorted" alone is satisfied by `fun _ => []`. "A permutation of the input" alone is satisfied by the identity. Together they pin down the result completely: we will *prove* that there is only one sorted permutation of a list. Writing a specification is mostly the art of noticing which programs it still allows.

31.2 Induction, revisited

Proofs about recursive programs are proofs by induction, and you have written many of them since Chapter 1. Two situations need more care than the ones we have met so far. The code is in `code/lean/Book/Ch31/Induction.lean`.

Generalizing the induction hypothesis

Reversing a list by repeatedly appending is quadratic. The standard fix is an accumulator, which also makes the function tail-recursive:

```
/-- Reverse with an accumulator: tail-recursive, and linear time. -/
def revAcc : List α → List α → List α
  | [], acc => acc
  | x :: xs, acc => revAcc xs (x :: acc)

def fastReverse (xs : List α) : List α := revAcc xs []
```

We want to prove `fastReverse xs = xs.reverse`, that is, `revAcc xs [] = xs.reverse`. The obvious induction gets stuck:

```
/--
error: unsolved goals
case cons
α : Type
x : α
xs : List α
ih : revAcc xs [] = xs.reverse
```

```
⊢ revAcc xs [x] = xs.reverse ++ [x]
-/
#guard_msgs in
theorem fastReverse_eq_attempt (xs : List α) : revAcc xs [] = xs.reverse := by
  induction xs with
  | nil => rfl
  | cons x xs ih => simp only [revAcc, List.reverse_cons]
```

Look at the goal in the `cons` case. After one step, `revAcc (x :: xs) []` became `revAcc xs [x]`: the recursive call has a *different* accumulator. But the induction hypothesis `ih` talks only about `revAcc xs []`, with the empty accumulator, so it does not apply. The hypothesis is too weak, because the statement we are proving is too specific.

The cure is to prove a *more general* statement, one that holds for every accumulator, and to tell `induction` to keep the accumulator universally quantified in the hypothesis:

```
/-- The general statement: for *every* accumulator. -/
theorem revAcc_eq (xs acc : List α) : revAcc xs acc = xs.reverse ++ acc := by
  induction xs generalizing acc with
  | nil =>
    -- goal: revAcc [] acc = [].reverse ++ acc
    rfl
  | cons x xs ih =>
    -- ih   : ∀ (acc : List α), revAcc xs acc = xs.reverse ++ acc
    -- goal : revAcc (x :: xs) acc = (x :: xs).reverse ++ acc
    rw [revAcc, ih (x :: acc)]
    -- goal : xs.reverse ++ x :: acc = (x :: xs).reverse ++ acc
    simp

theorem fastReverse_eq (xs : List α) : fastReverse xs = xs.reverse := by
  rw [fastReverse, revAcc_eq, List.append_nil]
```

`induction xs generalizing acc` makes the hypothesis `ih : ∀ (acc : List α), revAcc xs acc = xs.reverse ++ acc`, a statement about *every* accumulator. In the `cons` case we unfold `revAcc` one step and use `ih` with the accumulator `x :: acc`, the one the recursive call actually uses. What remains, `xs.reverse ++ x :: acc = (x :: xs).reverse ++ acc`, is list algebra that `simp` knows (`(x :: xs).reverse` is `xs.reverse ++ [x]`, and append is associative). Then `fastReverse_eq` is the special case `acc = []`.

The rule of thumb: when a recursive call changes an argument other than the one you are recursing on, the theorem must quantify over that argument, and the induction must generalize it. You will meet this again in the binary search, where both ends of the search interval change.

Induction on trees

Induction works on any inductive type, not just lists. For a binary tree, the induction principle has one case per constructor, and the `node` case gives two hypotheses, one for each child:

```
/-- Binary trees with a value at every node. -/
inductive Tree (α : Type) where
  | leaf : Tree α
  | node : Tree α → α → Tree α → Tree α

namespace Tree

def size : Tree α → ℕ
  | leaf => 0
```

```
  | node l _ r => l.size + 1 + r.size

/-- The elements from left to right. -/
def toList : Tree α → List α
  | leaf => []
  | node l x r => l.toList ++ [x] ++ r.toList

/-- Swap the children of every node. -/
def mirror : Tree α → Tree α
  | leaf => leaf
  | node l x r => node r.mirror x l.mirror
```

```
/-- The induction principle for trees gives two hypotheses, one per child. -/
theorem mirror_mirror (t : Tree α) : t.mirror.mirror = t := by
  induction t with
  | leaf => rfl
  | node l x r ihl ihr =>
    -- ihl : l.mirror.mirror = l,  ihr : r.mirror.mirror = r
    rw [mirror, mirror, ihl, ihr]

theorem length_toList (t : Tree α) : t.toList.length = t.size := by
  induction t with
  | leaf => rfl
  | node l x r ihl ihr =>
    simp only [toList, size, List.length_append, List.length_singleton, ihl, ihr]

@[simp] theorem toList_mirror (t : Tree α) : t.mirror.toList = t.toList.reverse := by
  induction t with
  | leaf => rfl
  | node l x r ihl ihr =>
    simp [mirror, toList, ihl, ihr]
```

In `mirror_mirror`, the `node` case has `ihl : l.mirror.mirror = l` and `ihr : r.mirror.mirror = r`. Unfolding `mirror` twice turns the goal `(node l x r).mirror.mirror = node l x r` into `node l.mirror.mirror x r.mirror.mirror = node l x r`, and the two hypotheses finish it. In `toList_mirror`, the in-order traversal of a mirrored tree is the reverse of the traversal of the tree; after unfolding, `simp` uses the hypotheses and the library fact that reversing `a ++ b` gives `b.reverse ++ a.reverse`.

Simp lemmas

`toList_mirror` carries the attribute `@[simp]`. That adds it to the set of equations that `simp` uses automatically, oriented from left to right, so any later goal mentioning `t.mirror.toList` gets rewritten to `t.toList.reverse` without being asked. Chapter 30 did the same with the entry lemmas of its matrices. The variant `simp?` runs `simp` and then reports which lemmas it used:

```
/-- Because `toList_mirror` is tagged `@[simp]`, `simp` uses it on its own. -/
example (t : Tree α) : t.mirror.mirror.toList = t.toList := by
  simp

/--
info: Try this:
  [apply] simp only [toList_mirror, List.reverse_reverse]
-/
#guard_msgs in
example (t : Tree α) : t.mirror.mirror.toList = t.toList := by
  simp?
```

The suggestion, `simp only [toList_mirror, List.reverse_reverse]`, is a *closed* version of the same step: it uses exactly those two lemmas and nothing else. A plain `simp` can change behavior when the library

adds new simp lemmas; `simp only` cannot, so it is the better choice in the middle of a long proof. Three guidelines for tagging your own lemmas `@[simp]`: the left side should be more complicated than the right side (so rewriting makes progress); the lemma should not loop (never tag commutativity, `a + b = b + a`, which `simp` could apply forever); and it should be a fact you want applied *every* time, such as how to compute an operation on a constructor. Related forms: `simp at h` simplifies a hypothesis, `simp [h]` adds `h` to the set for one call, and `simp_all` simplifies everything in sight.

31.3 Termination

Every Lean function must terminate. Chapter 2 introduced the rule and Chapter 30 showed one reason for it: type checking runs programs, so a non-terminating program could make the type checker loop. The other reason is logic. If Lean accepted `def loop : False := loop`, then `loop` would be a proof of `False`, and every theorem would be provable. The code for this section is in `code/lean/Book/Ch31/Termination.lean`.

Structural and well-founded recursion

The easy case is **structural recursion**: every recursive call is on a piece of the argument, as in Chapter 1's `sumTo`.

```
/-- Structural recursion: the recursive call is on `xs`, a piece of the
argument `x :: xs`. Lean accepts this without further ado. -/
def total : List ℕ → ℕ
  | [] => 0
  | x :: xs => x + total xs
```

Many natural functions are not structural. Halving a number calls itself on `n / 2`, which is smaller than `n` but not a piece of it:

```
/-- The number of times `n` can be halved before it drops below 2. The call is
on `n / 2`, which is not a piece of `n`, so the recursion is not structural. -/
def log2 (n : ℕ) : ℕ := if n < 2 then 0 else 1 + log2 (n / 2)
```

Lean accepts this anyway, by **well-founded recursion**. It looks for a *measure*, a function from the arguments to a type with a well-founded order (usually `ℕ` with `<`, where every decreasing sequence is finite), that decreases at every recursive call. Here the measure is `n` itself: in the branch that recurses, `n ≥ 2`, so `n / 2 < n`. Lean found the measure and proved the decrease on its own; `termination_by?` asks it to show what it found:

```
/--
info: Try this:
  [apply] termination_by n
-/
#guard_msgs in
def log2' (n : ℕ) : ℕ := if n < 2 then 0 else 1 + log2' (n / 2)
termination_by?
```

The proof that the measure decreases is produced by a tactic called `decreasing_tactic`, which tries simple arithmetic (including `omega`) with the facts known at the call site, such as the negated condition `¬n < 2`.

When Lean needs help

Euclid's algorithm calls itself on `a % b`:

```
/--
error: fail to show termination for
  Ch31.gcdBad
with errors
failed to infer structural recursion:
Cannot use parameter a:
  failed to eliminate recursive application
    gcdBad b (a % b)
Cannot use parameter b:
  failed to eliminate recursive application
    gcdBad b (a % b)


Could not find a decreasing measure.
The basic measures relate at each recursive call as follows:
(<, ≤, =: relation proved, ? all proofs failed, _: no proof attempted)
            a b
1) 61:49-65 ? ?
Please use `termination_by` to specify a decreasing measure.
-/
#guard_msgs in
def gcdBad (a b : ℕ) : ℕ := if b = 0 then a else gcdBad b (a % b)
```

Read the error from the top. Lean first tried structural recursion on each parameter and failed: `b` is not a piece of `a`, and `a % b` is not a piece of `b`. Then it tried the parameters as measures: the table has one row per recursive call (one call, at line 61) and one column per measure (`a`, `b`), and a `?` means that it could not prove the measure decreases. For `b` the fact is true (`a % b < b` whenever `b ≠ 0`), but proving it needs a lemma about `%` that the automatic tactic does not try. So we say which measure to use, and prove the decrease ourselves:

```
/-- Euclid's algorithm. The measure is `b`; the proof that it decreases is
`a % b < b`, which holds because `b ≠ 0` in the branch that recurses. -/
def euclid (a b : ℕ) : ℕ := if b = 0 then a else euclid b (a % b)
termination_by b
decreasing_by
  -- goal: a % b < b, with `h✝ : ¬b = 0` in the context
  apply Nat.mod_lt     -- `Nat.mod_lt : y > 0 → x % y < y`; remaining goal: b > 0
  omega                -- follows from `¬b = 0`
```

`termination_by b` names the measure. `decreasing_by` replaces the automatic proof with ours. The goal is `a % b < b`, and the context contains the negated condition of the `if` (Lean adds it automatically, as an anonymous hypothesis). `apply Nat.mod_lt` uses the library lemma `x % y < y` for `y > 0`, leaving the goal `b > 0`, and `omega` proves it from `¬b = 0`.

```
/-- info: (6, 1, 21) -/
#guard_msgs in
#eval (euclid 48 18, euclid 17 5, euclid 0 21)
```

Some functions need a measure with more than one component. In Ackermann's function, the first argument sometimes stays the same while the second grows enormously. What decreases is the *pair*, in lexicographic order: either the first component decreases, or it stays the same and the second decreases.

```
/-- Ackermann's function: neither argument decreases on every call, but the
pair does, in lexicographic order. -/
```

```
def ack : ℕ → ℕ → ℕ
  | 0, n => n + 1
  | m + 1, 0 => ack m 1
  | m + 1, n + 1 => ack m (ack (m + 1) n)
termination_by m n => (m, n)
```

```
/-- info: [3, 5, 5, 61] -/
#guard_msgs in
#eval [ack 1 1, ack 2 1, ack 3 0, ack 3 3]
```

What `partial` costs

Sometimes no measure exists, or nobody knows one. Whether the Collatz process (halve even numbers, map odd `n` to `3n + 1`) reaches 1 from every start is a famous open problem, so no termination proof can be given. `partial` tells Lean to skip the check:

```
/-- Nobody knows whether this terminates for every `n` (the Collatz
conjecture), so no measure can be given. `partial` skips the check. -/
partial def collatz (n : ℕ) : ℕ :=
  if n ≤ 1 then 0
  else if n % 2 = 0 then 1 + collatz (n / 2)
  else 1 + collatz (3 * n + 1)

/-- info: 111 -/
#guard_msgs in
#eval collatz 27
```

The function runs. But for the logic, a `partial` function is an opaque constant: Lean knows that it exists and has the right type (that is why a `partial` function's result type must be inhabited, Chapter 10), but not what it computes. Nothing can be proved about it, not even a fact that `#eval` confirms:

```
/--
error: Type mismatch
  rfl
has type
  ?m.7 = ?m.7
but is expected to have type
  collatz 6 = 8
-/
#guard_msgs in
example : collatz 6 = 8 := rfl
```

The way out is **fuel**: an extra argument that bounds the number of steps and decreases structurally. The function becomes total, returning `none` when the fuel runs out, and Lean can compute with it inside proofs:

```
/-- The same search with fuel: at most `fuel` steps, then give up with
`none`. Structural recursion on `fuel`, so the function is total, and Lean can
compute with it inside proofs. -/
def collatzFuel : ℕ → ℕ → Option ℕ
  | 0, _ => none
  | fuel + 1, n =>
    if n ≤ 1 then some 0
    else (collatzFuel fuel (if n % 2 = 0 then n / 2 else 3 * n + 1)).map (· + 1)

example : collatzFuel 10 6 = some 8 := by decide
example : collatzFuel 5 6 = none := by decide
```

(Chapter 7 used fuel the same way for the normalizer of the λ-calculus.) The price is that a theorem about `collatzFuel` must say something about the fuel, typically "if the result is `some k`, then ...".

Proving things about well-founded definitions

A function defined by well-founded recursion comes with its own induction principle, which follows the function's recursive structure:

```
/--
info: Ch31.log2.induct (motive : ℕ → Prop) (case1 : ∀ (x : ℕ), x < 2 → motive x)
  (case2 : ∀ (x : ℕ), ¬x < 2 → motive (x / 2) → motive x) (n : ℕ) : motive n
-/
#guard_msgs in
#check log2.induct
```

To prove `motive n` for every `n`, it is enough to prove it in the base case (`x < 2`) and, in the recursive case, to prove it for `x` assuming it for `x / 2`. This is **functional induction**, and the tactic `fun_induction` uses it. It also unfolds the function in the goal, case by case:

```
/-- `log2` really is a logarithm: `2 ^ log2 n ≤ n` for every positive `n`. -/
theorem pow_log2_le (n : ℕ) (h : 1 ≤ n) : 2 ^ log2 n ≤ n := by
  fun_induction log2 n with
  | case1 n hlt =>
    -- `log2 n` was unfolded to `0`; goal: 2 ^ 0 ≤ n
    simp only [pow_zero]
    exact h
  | case2 n hge ih =>
    -- ih   : 1 ≤ n / 2 → 2 ^ log2 (n / 2) ≤ n / 2
    -- goal : 2 ^ (1 + log2 (n / 2)) ≤ n
    calc 2 ^ (1 + log2 (n / 2))
        = 2 * 2 ^ log2 (n / 2) := by ring
      _ ≤ 2 * (n / 2) := by
          have h2 := ih (by omega)   -- n ≥ 2, so n / 2 ≥ 1
          omega
      _ ≤ n := by omega
```

In the first case `log2 n` has become `0`, so the goal is `2 ^ 0 ≤ n`; `simp only [pow_zero]` turns `2 ^ 0` into `1`, and the hypothesis `h : 1 ≤ n` closes it. The second case is a `calc` block, a proof written as a chain of relations, each with its own justification:

- `2 ^ (1 + log2 (n / 2)) = 2 * 2 ^ log2 (n / 2)` by `ring`, which proves equations in commutative rings and knows how exponents add;
- `_ ≤ 2 * (n / 2)` by the induction hypothesis, which needs `1 ≤ n / 2` (true, because `n ≥ 2`) and which `omega` then scales by 2;
- `_ ≤ n` by `omega`, which knows how `/` by a literal behaves.

The underscore `_` at the start of each line stands for the right side of the previous line. `calc` lets a proof read the way you would write it on paper, and Lean chains the steps with transitivity.

31.4 A verified insertion sort

We now have the tools for the main example. The code is in `code/lean/Book/Ch31/Sorting.lean`.

```
/-- Insert `x` into a list, before the first element that is at least `x`.
If the list is sorted, so is the result. -/
def insertSorted (x : ℤ) : List ℤ → List ℤ
  | [] => [x]
  | y :: ys => if x ≤ y then x :: y :: ys else y :: insertSorted x ys

/-- Sort the tail, then insert the head. -/
```

```
def isort : List ℤ → List ℤ
  | [] => []
  | x :: xs => insertSorted x (isort xs)
```

```
/-- info: [-3, 1, 2, 5, 5, 9] -/
#guard_msgs in
#eval isort [5, 2, 9, -3, 5, 1]
```

The specification, in Mathlib's vocabulary

For "is a permutation of" we use `List.Perm` from Lean's core library, written `l₁ ~ l₂` (the notation lives in the `List` namespace, so the file opens it with `open scoped List`). It is an inductive relation with four constructors:

```
/--
info: inductive List.Perm.{u} : {α : Type u} → List α → List α → Prop
number of parameters: 1
constructors:
List.Perm.nil : ∀ {α : Type u}, [] ~ []
List.Perm.cons : ∀ {α : Type u} (x : α) {l₁ l₂ : List α}, l₁ ~ l₂ → x :: l₁ ~ x :: l₂
List.Perm.swap : ∀ {α : Type u} (x y : α) (l : List α), y :: x :: l ~ x :: y :: l
List.Perm.trans : ∀ {α : Type u} {l₁ l₂ l₃ : List α}, l₁ ~ l₂ → l₂ ~ l₃ → l₁ ~ l₃
-/
#guard_msgs in
#print List.Perm
```

Two empty lists are permutations of each other; adding the same head preserves the relation; swapping the first two elements is a permutation; and permutations compose. Every permutation of a list can be built from these four rules.

For "is sorted" we use Mathlib's `List.SortedLE`. It is defined as "`l.get` is a monotone function", and Mathlib proves it equivalent to the inductive form `List.Pairwise (· ≤ ·)`: every element is at most every later element. Our proofs use the `Pairwise` form, which is easier to do induction on, and convert at the end. Sortedness of a concrete list is decidable:

```
example (l : List ℤ) : l.SortedLE ↔ l.Pairwise (· ≤ ·) := List.sortedLE_iff_pairwise

example : [1, 2, 2, 7].SortedLE := by decide
example : ¬ [1, 3, 2].SortedLE := by decide
```

> **NOTE**
>
> Older versions of Mathlib, and much of the material you will find online, use `List.Sorted r l`, with an explicit relation argument. In the Mathlib version pinned by this book, sortedness is expressed with `List.SortedLE`, `List.SortedLT` (and their duals) or directly with `List.Pairwise`. If a name from an older text does not exist, search `.lake/packages/mathlib/Mathlib/Data/List/Sort.lean` for its replacement.

Insertion produces a permutation

```
/-- Inserting `x` gives a permutation of `x :: l`. -/
theorem perm_insertSorted (x : ℤ) (l : List ℤ) : insertSorted x l ~ x :: l := by
  induction l with
  | nil => exact List.Perm.refl [x]
```

```
  | cons y ys ih =>
    -- ih : insertSorted x ys ~ x :: ys
    rw [insertSorted]
    split
    · -- case x ≤ y: the result is literally x :: y :: ys
      exact List.Perm.refl _
    · -- case ¬ x ≤ y: goal y :: insertSorted x ys ~ x :: y :: ys
      calc y :: insertSorted x ys
          ~ y :: x :: ys := List.Perm.cons y ih
        _ ~ x :: y :: ys := List.Perm.swap x y ys

theorem mem_insertSorted {x z : ℤ} {l : List ℤ} :
    z ∈ insertSorted x l ↔ z = x ∨ z ∈ l := by
  rw [(perm_insertSorted x l).mem_iff, List.mem_cons]
```

The proof is by induction on `l`. In the `nil` case, `insertSorted x []` is `[x]`, and every list is a permutation of itself (`List.Perm.refl`). In the `cons y ys` case, `rw [insertSorted]` unfolds one step, which exposes the `if`, and `split` makes one goal for each branch. When `x ≤ y`, the result is literally `x :: y :: ys`, so reflexivity again. When `x > y`, the result is `y :: insertSorted x ys`, and a `calc` with `~` instead of `≤` chains two permutations: the induction hypothesis under the head `y` (`List.Perm.cons`), then a swap of the first two elements (`List.Perm.swap`). `calc` works with any relation that has a transitivity instance, and Lean's library provides one for `~`.

`mem_insertSorted` follows at once: permutations have the same members (`Perm.mem_iff`), and membership in `x :: l` means being `x` or being in `l`.

Insertion preserves sortedness

This is the longest proof of the chapter, and the most typical: a case analysis that mirrors the program, with a small argument in each case.

```
/-- Inserting into a sorted list gives a sorted list. -/
theorem sorted_insertSorted (x : ℤ) (l : List ℤ) (h : l.Pairwise (· ≤ ·)) :
    (insertSorted x l).Pairwise (· ≤ ·) := by
  induction l with
  | nil => simp [insertSorted]
  | cons y ys ih =>
    -- h : (y :: ys).Pairwise (· ≤ ·), which unfolds to two facts:
    rw [List.pairwise_cons] at h
    obtain ⟨hy, hys⟩ := h       -- hy : ∀ b ∈ ys, y ≤ b    hys : ys.Pairwise (· ≤ ·)
    rw [insertSorted]
    split
    · -- case x ≤ y: goal (x :: y :: ys).Pairwise (· ≤ ·)
      rename_i hxy
      rw [List.pairwise_cons, List.pairwise_cons]
      refine ⟨?_, hy, hys⟩
      -- goal: ∀ b ∈ y :: ys, x ≤ b
      intro b hb
      rw [List.mem_cons] at hb
      rcases hb with rfl | hb
      · exact hxy                       -- b = y
      · calc x ≤ y := hxy               -- b ∈ ys
            _ ≤ b := hy b hb
    · -- case ¬ x ≤ y: goal (y :: insertSorted x ys).Pairwise (· ≤ ·)
      rename_i hxy
      rw [List.pairwise_cons]
      refine ⟨?_, ih hys⟩
      -- goal: ∀ b ∈ insertSorted x ys, y ≤ b
      intro b hb
      rw [mem_insertSorted] at hb
      rcases hb with rfl | hb
```

```
      · omega                                  -- b = x, and ¬ x ≤ y
      · exact hy b hb
```

Step by step:

- `rw [List.pairwise_cons] at h` rewrites the hypothesis with the library equivalence `(a :: l).Pairwise R ↔ (∀ b ∈ l, R a b) ∧ l.Pairwise R`, and `obtain ⟨hy, hys⟩ := h` splits the conjunction into its two halves.
- `rw [insertSorted]` and `split` expose the two branches of the program, as before. `rename_i hxy` gives a name to the branch condition, which `split` introduced as an anonymous hypothesis.
- In the branch `x ≤ y` the goal is that `x :: y :: ys` is sorted. Rewriting with `pairwise_cons` twice turns it into three facts: `x` is at most every element of `y :: ys`, `y` is at most every element of `ys`, and `ys` is sorted. `refine ⟨?_, hy, hys⟩` supplies the last two from the hypotheses and leaves the first as a new goal (`?_` marks the hole). For an element `b` of `y :: ys`, either `b = y` (then `hxy` is the proof; `rcases hb with rfl | hb` substitutes `y` for `b`), or `b ∈ ys`, and a two-step `calc` chains `x ≤ y` and `y ≤ b`.
- In the branch `¬ x ≤ y` the result is `y :: insertSorted x ys`. The tail is sorted by the induction hypothesis `ih hys`. For the head, every element `b` of `insertSorted x ys` is either `x`, and then `y ≤ x` follows from `¬ x ≤ y` by `omega`, or an element of `ys`, and then `hy` applies.

Putting the two lemmas together gives the correctness of the sort:

```
theorem isort_perm (xs : List ℤ) : isort xs ~ xs := by
  induction xs with
  | nil => exact List.Perm.nil
  | cons x xs ih =>
    -- goal: insertSorted x (isort xs) ~ x :: xs
    calc insertSorted x (isort xs)
        ~ x :: isort xs := perm_insertSorted x (isort xs)
      _ ~ x :: xs := List.Perm.cons x ih

theorem isort_pairwise (xs : List ℤ) : (isort xs).Pairwise (· ≤ ·) := by
  induction xs with
  | nil => exact List.Pairwise.nil
  | cons x xs ih => exact sorted_insertSorted x (isort xs) ih

theorem isort_sorted (xs : List ℤ) : (isort xs).SortedLE :=
  (isort_pairwise xs).sortedLE
```

and the specification as a type:

```
/-- The specification as a type: a list, together with proofs that it is
sorted and that it is a permutation of the input. -/
def sort (xs : List ℤ) : { ys : List ℤ // ys.SortedLE ∧ ys ~ xs } :=
  ⟨isort xs, isort_sorted xs, isort_perm xs⟩
```

```
/-- info: [-3, 1, 2, 5, 5, 9] -/
#guard_msgs in
#eval (sort [5, 2, 9, -3, 5, 1]).val
```

`.val` extracts the list from the subtype; the two proofs travel with it during type checking and vanish when the program is compiled.

Letting `grind` do it

Chapter 1 used `grind` once, to finish Gauss's formula. It is Lean's most powerful general-purpose automation: it collects the facts in the context and the lemmas you give it, follows equalities (congruence closure), splits on `if`s and on disjunctions, instantiates the lemmas where they match, and does linear arithmetic. Given the definition and the two library facts we used by hand, it proves the sortedness lemma in one line:

```
/-- The same theorem as `sorted_insertSorted`, with the case analysis, the
membership reasoning and the arithmetic left to `grind`. -/
theorem sorted_insertSorted' (x : ℤ) (l : List ℤ) (h : l.Pairwise (· ≤ ·)) :
    (insertSorted x l).Pairwise (· ≤ ·) := by
  induction l with
  | nil => simp [insertSorted]
  | cons y ys ih => grind [insertSorted, List.pairwise_cons, mem_insertSorted]
```

When should you write the long version? When you want to understand the proof, when you want it to survive changes to the automation, and when `grind` fails and you need to see where. When should you use `grind`? When the argument is routine and you would rather read the theorem than the proof. (Like the `grind` proof in Chapter 1, it relies on classical reasoning; Chapter 5 explained what that means, and `#print axioms` shows it.)

The specification determines the output

We claimed that "sorted" and "a permutation" together pin down the result. Here is the proof, and two consequences:

```
/-- The specification determines the output: two sorted permutations of the
same list are equal. -/
theorem sorted_perm_unique {ys zs : List ℤ} (hy : ys.SortedLE) (hz : zs.SortedLE)
    (h : ys ~ zs) : ys = zs :=
  List.Perm.eq_of_sortedLE hy hz h

/-- So our sort agrees with Mathlib's insertion sort on every input... -/
theorem isort_eq_insertionSort (xs : List ℤ) :
    isort xs = xs.insertionSort (· ≤ ·) := by
  apply sorted_perm_unique (isort_sorted xs) List.sortedLE_insertionSort
  -- goal: isort xs ~ List.insertionSort (· ≤ ·) xs
  exact (isort_perm xs).trans (List.perm_insertionSort _ xs).symm

/-- ... and with the merge sort of Lean's core library. -/
theorem isort_eq_mergeSort (xs : List ℤ) : isort xs = xs.mergeSort := by
  apply sorted_perm_unique (isort_sorted xs) List.sortedLE_mergeSort
  exact (isort_perm xs).trans (List.mergeSort_perm xs _).symm
```

`sorted_perm_unique` is Mathlib's `List.Perm.eq_of_sortedLE`: two sorted lists that are permutations of each other are equal. So *any* function that meets our specification computes exactly the same function as `isort`. Mathlib's `List.insertionSort` meets it (Mathlib proves sortedness and permutation for it), and so does core Lean's `List.mergeSort`; therefore, for every input, `isort xs` equals both. We have proved two different library sorting algorithms equal to ours without looking at how they work, only at what they promise.

This fact will matter for testing. If the specification determines the output, then *comparing outputs* with a verified model is the same as checking the specification. When it does not, as we will see with binary search, comparing outputs can reject correct implementations.

How many comparisons?

The specification says nothing about cost. Cost is a separate property, and we can prove it separately. We count the comparisons that `insertSorted` and `isort` make, with functions that follow their recursion:

```
/-- The number of comparisons `insertSorted x l` makes. -/
def insertCost (x : ℤ) : List ℤ → ℕ
  | [] => 0
  | y :: ys => if x ≤ y then 1 else 1 + insertCost x ys

/-- The number of comparisons `isort` makes. -/
def isortCost : List ℤ → ℕ
  | [] => 0
  | x :: xs => insertCost x (isort xs) + isortCost xs
```

```
theorem insertCost_le (x : ℤ) (l : List ℤ) : insertCost x l ≤ l.length := by
  induction l with
  | nil => simp [insertCost]
  | cons y ys ih =>
    simp only [insertCost, List.length_cons]
    split <;> omega

/-- At most `n (n - 1) / 2` comparisons for a list of length `n`, stated
without subtraction or division. -/
theorem isortCost_le (xs : List ℤ) :
    2 * isortCost xs + xs.length ≤ xs.length * xs.length := by
  induction xs with
  | nil => simp [isortCost]
  | cons x xs ih =>
    -- one insertion into a list of length `xs.length`
    have hc : insertCost x (isort xs) ≤ xs.length := by
      have h := insertCost_le x (isort xs)
      rwa [(isort_perm xs).length_eq] at h
    rw [isortCost, List.length_cons]
    calc 2 * (insertCost x (isort xs) + isortCost xs) + (xs.length + 1)
        = (2 * isortCost xs + xs.length) + 2 * insertCost x (isort xs) + 1 := by ring
      _ ≤ xs.length * xs.length + 2 * xs.length + 1 := by linarith
      _ = (xs.length + 1) * (xs.length + 1) := by ring
```

`insertCost_le` is a routine induction: after unfolding, `split` creates one goal per branch of the `if`, and `<;> omega` closes both with the induction hypothesis.

`isortCost_le` says that the number of comparisons C for a list of length n satisfies $2C + n \le n^2$, that is, $C \le n(n-1)/2$, written without the subtraction and division that are awkward on `ℕ`. In the `cons` case the insertion costs at most `xs.length` comparisons (`hc`; note the use of `(isort_perm xs).length_eq`, since we insert into the *sorted* tail, whose length equals that of `xs`). The `calc` block then has three steps, each with the right tool:

- an equation that only rearranges terms, by `ring`;
- an inequality that follows *linearly* from `ih` and `hc`, by `linarith`;
- the polynomial identity $n^2 + 2n + 1 = (n+1)^2$, by `ring`.

The middle step is the reason for `linarith`. The goal and the hypotheses mention the product `xs.length * xs.length`, which is not linear arithmetic. `linarith` treats such a product as an opaque quantity and finds a linear combination of the hypotheses that implies the goal: here, `ih` plus twice `hc`. `omega` is the right tool for linear arithmetic with `/`, `%` and literals over `ℕ` and `ℤ`; `linarith`

for linear reasoning over ordered fields (and `ℕ` and `ℤ`), possibly with nonlinear atoms; `ring` for equations that hold as polynomial identities.

```
/-- info: (15, 5, 3) -/
#guard_msgs in
#eval (isortCost [6, 5, 4, 3, 2, 1], isortCost [1, 2, 3, 4, 5, 6], isortCost [1, 1, 1, 1])
```

The bound is attained: the reversed list of six elements needs $15 = 6 \cdot 5/2$ comparisons, and a sorted list needs only $n - 1$.

31.5 A verified binary search

Binary search is a famously easy algorithm to get subtly wrong, and a good test of the techniques so far: its recursion is not structural, and its correctness argument is an invariant about an interval. The code is in `code/lean/Book/Ch31/BinarySearch.lean`.

```
/-- Search for `x` in the index interval `[lo, hi)` of `a`. -/
def searchFrom (a : Array ℤ) (x : ℤ) (lo hi : ℕ) (hhi : hi ≤ a.size) : Option ℕ :=
  if h : lo < hi then
    let mid := lo + (hi - lo) / 2
    -- `a[mid]` below needs this fact; Lean finds it in the context.
    have hmid : mid < a.size := by omega
    if x < a[mid] then searchFrom a x lo mid (by omega)
    else if a[mid] < x then searchFrom a x (mid + 1) hi hhi
    else some mid
  else none
termination_by hi - lo

def search (a : Array ℤ) (x : ℤ) : Option ℕ :=
  searchFrom a x 0 a.size (Nat.le_refl _)
```

`searchFrom` looks for `x` in the index interval `[lo, hi)`. Its extra argument `hhi : hi ≤ a.size` is a proof, carried along so that the indexing `a[mid]` type-checks: from `lo < hi` and `hi ≤ a.size`, `omega` proves `mid < a.size`, and the `have` puts that fact in the context, where the indexing finds it (Chapter 30). The recursive calls must supply the proof for their own intervals, `mid ≤ a.size` and the unchanged `hhi`. The measure is the width of the interval, `hi - lo`, which strictly decreases on both recursive calls; Lean proves this on its own, from `lo < hi` and the definition of `mid`.

```
/-- info: [some 0, some 3, none, none, none] -/
#guard_msgs in
#eval [2, 11, 4, 1, 20].map (search #[2, 3, 7, 11, 13])
```

The midpoint is computed as `lo + (hi - lo) / 2` rather than `(lo + hi) / 2`. On Lean's `ℕ` the two are equal, since there is no overflow; we use the first form because the Rust implementation must, and it is better if the model and the implementation are the same algorithm. We return to this point below.

Soundness: a returned index is right

The first half of the specification does not need the array to be sorted:

```
/-- Soundness: a returned index is in bounds and points at `x`. -/
theorem searchFrom_some (a : Array ℤ) (x : ℤ) (lo hi : ℕ) (hhi : hi ≤ a.size) (i : ℕ) :
    searchFrom a x lo hi hhi = some i → ∃ h : i < a.size, a[i] = x := by
  fun_induction searchFrom a x lo hi hhi with
  | case1 lo hi hhi h mid hmid hlt ih =>
```

```
    -- went left: the goal is literally the induction hypothesis
    exact ih
  | case2 lo hi hhi h mid hmid hnlt hgt ih =>
    -- went right: the same
    exact ih
  | case3 lo hi hhi h mid hmid hnlt hngt =>
    -- found: the result is `some mid`
    intro hsome
    injection hsome with hmi        -- hmi : mid = i
    subst hmi
    exact ⟨hmid, by omega⟩          -- ¬ x < a[mid] and ¬ a[mid] < x give a[mid] = x
  | case4 lo hi hhi h =>
    -- empty interval: the result is `none`, which is not `some i`
    intro hnone
    cases hnone
```

The theorem is stated as an implication in the goal, `searchFrom … = some i → …`, rather than with the hypothesis before the colon, so that functional induction transforms it together with the function call. `fun_induction` produces one case for each way `searchFrom` can return, with the branch conditions as hypotheses and, in each case, the call replaced by what that branch returns:

- `case1`, the branch that recursed to the left: the goal is now about the recursive call `searchFrom a x lo mid _`, which is exactly the induction hypothesis `ih`.
- `case2`, the branch that recursed to the right: the same.
- `case3`, the branch that returned `some mid`. The hypothesis `hsome : some mid = some i` is decomposed by `injection` into `mid = i`, and `subst` replaces `i` by `mid`. What is left is to show `mid < a.size` (that is `hmid`) and `a[mid] = x`, which `omega` derives from the two failed comparisons `¬ x < a[mid]` and `¬ a[mid] < x`. (`omega` treats `a[mid]` as an unknown integer.)
- `case4`, the empty interval: the function returned `none`, and `none = some i` is impossible; `cases` on it finds no constructor that fits and closes the goal.

Completeness: `none` means absent

The second half needs sortedness. We state it for arrays directly:

```
/-- Sorted, for arrays: an element is never larger than a later one. -/
def SortedArr (a : Array ℤ) : Prop :=
  ∀ (i j : ℕ) (hi : i < a.size) (hj : j < a.size), i ≤ j → a[i] ≤ a[j]
```

```
/-- Completeness: on a sorted array, `none` means that no index in `[lo, hi)`
holds `x`. -/
theorem searchFrom_none (a : Array ℤ) (hs : SortedArr a) (x : ℤ) (lo hi : ℕ)
    (hhi : hi ≤ a.size) :
    searchFrom a x lo hi hhi = none →
      ∀ (k : ℕ) (hk : k < a.size), lo ≤ k → k < hi → a[k] ≠ x := by
  fun_induction searchFrom a x lo hi hhi with
  | case1 lo hi hhi h mid hmid hlt ih =>
    -- x < a[mid]: we searched [lo, mid); everything from mid on is too big
    intro hnone k hk hlo hkhi
    by_cases hkm : k < mid
    · exact ih hnone k hk hlo hkm
    · have hle : a[mid] ≤ a[k] := hs mid k hmid hk (by omega)
      omega                          -- x < a[mid] ≤ a[k], so a[k] ≠ x
  | case2 lo hi hhi h mid hmid hnlt hgt ih =>
    -- a[mid] < x: we searched [mid + 1, hi); everything up to mid is too small
    intro hnone k hk hlo hkhi
    by_cases hkm : mid < k
    · exact ih hnone k hk (by omega) hkhi
```

```
    · have hle : a[k] ≤ a[mid] := hs k mid hk hmid (by omega)
      omega                          -- a[k] ≤ a[mid] < x
  | case3 lo hi hhi h mid hmid hnlt hngt =>
    -- found: `some mid` is not `none`
    intro hnone
    cases hnone
  | case4 lo hi hhi h =>
    -- empty interval: there is no k with lo ≤ k < hi
    intro _ k hk hlo hkhi
    omega
```

The statement carries the invariant of the algorithm: if the search of `[lo, hi)` returns `none`, then no index `k` in `[lo, hi)` holds `x`. This is the generalization that the `revAcc` example taught us to look for: both ends of the interval change during the recursion, so the theorem is about an arbitrary interval, and the top-level search is the special case `[0, a.size)`. In `case1` the search went left, because `x < a[mid]`. For an index `k` below `mid`, the induction hypothesis applies. For `k ≥ mid`, sortedness gives `a[mid] ≤ a[k]`, so `x < a[k]`, and `omega` concludes `a[k] ≠ x`. `case2` is the mirror image. `case3` is impossible (the function returned `some mid`), and in `case4` the interval is empty, so `lo ≤ k < hi` is contradictory and `omega` says so.

The whole specification

```
theorem search_some {a : Array ℤ} {x : ℤ} {i : ℕ} (h : search a x = some i) :
    ∃ h : i < a.size, a[i] = x :=
  searchFrom_some a x 0 a.size _ i h

theorem search_none {a : Array ℤ} {x : ℤ} (hs : SortedArr a) (h : search a x = none) :
    x ∉ a := by
  intro hx
  obtain ⟨k, hk, hkx⟩ := Array.mem_iff_getElem.mp hx
  exact searchFrom_none a hs x 0 a.size _ h k hk (Nat.zero_le k) hk hkx

/-- The whole specification in one line, for sorted arrays. -/
theorem search_isSome_iff {a : Array ℤ} {x : ℤ} (hs : SortedArr a) :
    (search a x).isSome ↔ x ∈ a := by
  constructor
  · intro h
    obtain ⟨i, hi⟩ := Option.isSome_iff_exists.mp h
    obtain ⟨hlt, heq⟩ := search_some hi
    exact Array.mem_iff_getElem.mpr ⟨i, hlt, heq⟩
  · intro hx
    cases hr : search a x with
    | none => exact absurd hx (search_none hs hr)
    | some i => rfl
```

`search_none` converts the statement about indices into one about membership with the library lemma `Array.mem_iff_getElem`. `search_isSome_iff` is the specification in one line: on a sorted array, the search succeeds exactly when the key is present. As with the sort, we can package it as a type:

```
/-- The specification as a type, once more. -/
def searchChecked (a : Array ℤ) (hs : SortedArr a) (x : ℤ) :
    { r : Option ℕ //
      (∀ i, r = some i → ∃ h : i < a.size, a[i] = x) ∧ (r = none → x ∉ a) } :=
  ⟨search a x, fun _ h => search_some h, fun h => search_none hs h⟩
```

Notice what this specification does *not* say: which index is returned when the key occurs more than once. Any index holding `x` satisfies it. That freedom is deliberate, since it lets implementations choose, but it will matter when we test.

Connecting the pieces

Our `SortedArr` is a property of arrays; Mathlib's `SortedLE` is a property of lists. A short bridge connects them, and with it we can search the output of the verified sort:

```
/-- Mathlib's sortedness of the underlying list gives ours. -/
theorem sortedArr_of_sortedLE {a : Array ℤ} (h : a.toList.SortedLE) : SortedArr a := by
  intro i j hi hj hij
  have hl := List.sortedLE_iff_getElem_le_getElem_of_le.mp h
    (i := i) (j := j) (hi := by simpa using hi) (hj := by simpa using hj) hij
  simpa using hl

/-- Searching the output of the verified sort finds exactly the input's elements. -/
theorem search_isort (xs : List ℤ) (x : ℤ) :
    (search (isort xs).toArray x).isSome ↔ x ∈ xs := by
  have hs : SortedArr (isort xs).toArray :=
    sortedArr_of_sortedLE (by simpa using isort_sorted xs)
  rw [search_isSome_iff hs, List.mem_toArray]
  exact (isort_perm xs).mem_iff
```

The bridge uses Mathlib's characterization of `SortedLE` by indices, `List.sortedLE_iff_getElem_le_getElem_of_le`, and `simpa` to translate between `a.toList[i]` and `a[i]` (`simpa using e` simplifies both the goal and the expression `e` and then closes the goal with it). `search_isort` combines three results: the sort's output is sorted, the search is complete and sound on sorted arrays, and a permutation has the same members. Finally, here is what the whole development rests on:

```
/-- info: 'Ch31.search_isort' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms search_isort
```

The three standard axioms of Lean's logic (Chapter 1), and nothing else: no `sorry`, no unchecked assumption.

31.6 Running verified code

The functions we verified are ordinary programs. There are two ways to run them, and the difference matters:

```
/-- The kernel can run the sort inside a proof: slow, but trusted. -/
example : isort [3, 1, 2] = [1, 2, 3] := by decide

/-- The numbers below 1009 that `n * 7919 % 1009` hits for `n < 1000`, in a
scrambled order. `#eval` runs the sort through Lean's compiler: fast, but
outside the kernel. -/
def scrambled : List ℤ := (List.range 1000).map fun n => (((n * 7919) % 1009 : ℕ) : ℤ)

/-- info: (1000, [0, 1, 2, 3, 4]) -/
#guard_msgs in
#eval ((sort scrambled).val.length, (sort scrambled).val.take 5)
```

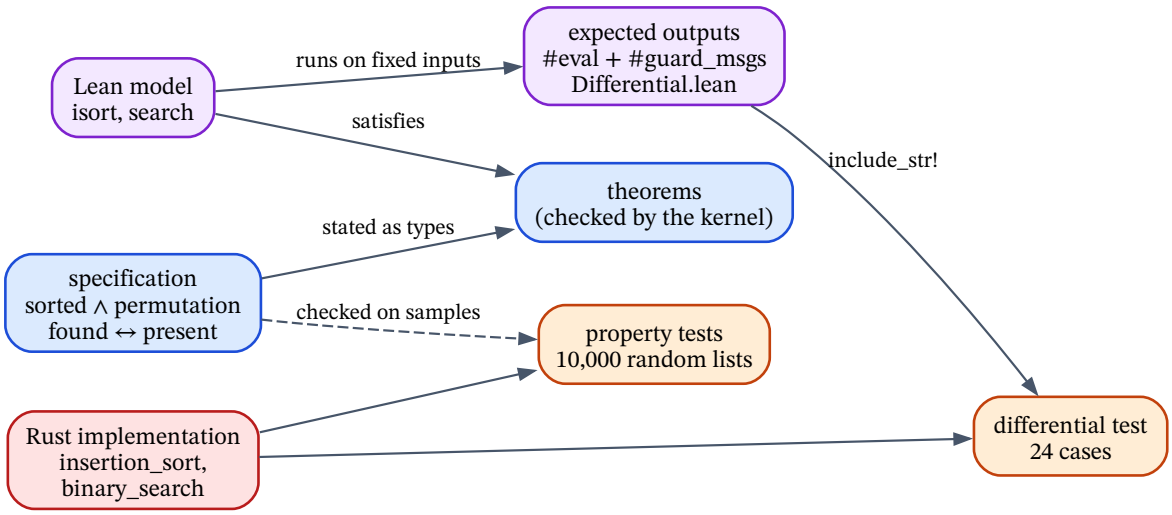
`decide` runs the sort *inside the kernel*, the small trusted checker at the heart of Lean, by reducing the terms step by step. That is slow, but the result is a proof that the kernel itself has checked. `#eval` compiles the program with Lean's code generator and runs it, outside the kernel: fast enough to sort a thousand numbers with a quadratic algorithm in a blink, but the result is trusted only as much as

the compiler. (There is a tactic, `native_decide`, that uses compiled code to prove propositions; it adds the compiler to the trusted base, and this book does not use it.)

The proofs inside the subtype returned by `sort` cost nothing at run time: Lean erases proofs when it compiles, so `(sort xs).val` runs exactly the code of `isort`. Verified Lean code can also be compiled into a native executable with Lake. For this chapter we need something different: not to ship the Lean code, but to use it as a reference for code written in Rust.

31.7 Putting it together: model in Lean, implement in Rust

Here is the workflow, as a diagram:



Model in Lean, implement in Rust. The specification is proved about the Lean model; the model's outputs on fixed inputs are pinned by `#guard_msgs` and read by the Rust tests; property tests check the same specification on random inputs.

Step 1: the model produces expected outputs

`code/lean/Book/Ch31/Differential.lean` runs the verified models on a fixed set of inputs. Some inputs are edge cases chosen by hand; the others come from a small pseudo-random generator, so that they are varied but reproducible:

```
/-- A linear congruential generator: reproducible pseudo-random numbers. -/
def lcgNext (s : ℕ) : ℕ := (s * 1103515245 + 12345) % 2147483648

/-- `n` pseudo-random integers between -9 and 9, starting from seed `s`. -/
def randomList : ℕ → ℕ → List ℤ
  | _, 0 => []
  | s, n + 1 =>
    let s' := lcgNext s
    let x : ℤ := ((s' / 65536 % 19 : ℕ) : ℤ) - 9
    x :: randomList s' n

/-- Edge cases first, then pseudo-random lists with many duplicates. -/
def sortInputs : List (List ℤ) :=
  [[], [5], [2, 1], [4, 3, 2, 1], [5, -2, 5, 0, -2]] ++
    (List.range 4).map fun k => randomList (k + 1) (k + 6)
```

```
/-- Keys to look for in a sorted list: its smallest element (often repeated),
zero (sometimes present) and ten (never present). -/
def searchKeys (ys : List ℤ) : List ℤ := [ys.headD 0, 0, 10]
```

Each test case becomes one line of text:

```
def showResult : Option ℕ → String
  | some i => s!"some {i}"
  | none => "none"

def sortLine (xs : List ℤ) : String :=
  s!"sort {xs} = {(sort xs).val}"

def searchLine (xs : List ℤ) (x : ℤ) : String :=
  s!"search {xs} {x} = {showResult (search xs.toArray x)}"

/-- Every sort case, then every key in the sorted random lists. -/
def table : List String :=
  sortInputs.map sortLine ++
    ((sortInputs.drop 4).map isort).flatMap fun ys => (searchKeys ys).map (searchLine ys)
```

And the table is printed and pinned:

```
/--
info: sort [] = []
sort [5] = [5]
sort [2, 1] = [1, 2]
sort [4, 3, 2, 1] = [1, 2, 3, 4]
sort [5, -2, 5, 0, -2] = [-2, -2, 0, 5, 5]
sort [-5, -8, -4, 7, -4, -6] = [-8, -6, -5, -4, -4, 7]
sort [6, 8, 8, 4, 7, -8, 4] = [-8, 4, 4, 6, 7, 8, 8]
sort [-8, -8, 1, 0, -2, -2, 3, -8] = [-8, -8, -8, -2, -2, 0, 1, 3]
sort [3, 8, -6, -3, 8, 3, -5, 1, -8] = [-8, -6, -5, -3, 1, 3, 3, 8, 8]
search [-2, -2, 0, 5, 5] -2 = some 1
search [-2, -2, 0, 5, 5] 0 = some 2
search [-2, -2, 0, 5, 5] 10 = none
search [-8, -6, -5, -4, -4, 7] -8 = some 0
search [-8, -6, -5, -4, -4, 7] 0 = none
search [-8, -6, -5, -4, -4, 7] 10 = none
search [-8, 4, 4, 6, 7, 8, 8] -8 = some 0
search [-8, 4, 4, 6, 7, 8, 8] 0 = none
search [-8, 4, 4, 6, 7, 8, 8] 10 = none
search [-8, -8, -8, -2, -2, 0, 1, 3] -8 = some 2
search [-8, -8, -8, -2, -2, 0, 1, 3] 0 = some 5
search [-8, -8, -8, -2, -2, 0, 1, 3] 10 = none
search [-8, -6, -5, -3, 1, 3, 3, 8, 8] -8 = some 0
search [-8, -6, -5, -3, 1, 3, 3, 8, 8] 0 = none
search [-8, -6, -5, -3, 1, 3, 3, 8, 8] 10 = none
-/
#guard_msgs in
#eval for line in table do IO.println line
```

This is where `#guard_msgs` earns its keep. The doc comment is the table, and Lean checks, every time the file is built, that the model still prints exactly this table. If someone changed `isort` or `search` in a way that changed an output, the Lean build would fail here, even if the change still satisfied the theorems. The table was produced by running the file once without the doc comment, reading the output, and pasting it in, the same procedure the book uses for every Lean output it shows.

Step 2: the Rust implementation

The Rust code (`code/rust/examples/ch31_sort.rs`) is written the way a Rust programmer would write it, not as a transliteration of the Lean model. The sort works in place on a slice:

```rust
/// Insertion sort, in place: the usual imperative formulation. Each new
/// element is swapped leftwards until the prefix `v[..=i]` is sorted again.
fn insertion_sort(v: &mut [i64]) {
    for i in 1..v.len() {
        let mut j = i;
        while j > 0 && v[j - 1] > v[j] {
            v.swap(j - 1, j);
            j -= 1;
        }
    }
}
```

This is a different algorithm from `isort`: it inserts each element into the sorted prefix on its left, by swapping, while `isort` inserts the head into the sorted tail on its right, building a new list. Because the specification determines the output, the two must agree anyway. The search is a loop with the same midpoint as the model:

```rust
/// Binary search on a sorted slice: the same algorithm as the Lean model
/// `searchFrom`, with a loop instead of tail recursion. Invariant: if `x` is
/// in the slice, it is at an index in `lo..hi`.
fn binary_search(a: &[i64], x: i64) -> Option<usize> {
    let (mut lo, mut hi) = (0, a.len());
    while lo < hi {
        // Not `(lo + hi) / 2`: that sum can overflow; this cannot, since lo < hi.
        let mid = lo + (hi - lo) / 2;
        if x < a[mid] {
            hi = mid;
        } else if a[mid] < x {
            lo = mid + 1;
        } else {
            return Some(mid);
        }
    }
    None
}
```

The midpoint matters. In Rust, `lo + hi` can overflow when the slice is large enough, while `lo + (hi - lo) / 2` cannot, because `lo < hi`. A bug of exactly this kind was found in 2006 in the binary search of Java's standard library: the algorithm was right, but the sum of the two indices overflowed on very large arrays. Our Lean proof works with `ℕ`, which has no overflow, so it *cannot* see this problem: it proves the algorithm correct, not the Rust arithmetic. Using the overflow-free form in both languages keeps the model and the implementation the same algorithm.

Step 3: the differential test

The Rust program reads the Lean file at compile time with `include_str!`, which embeds a file's contents as a `&str`, and extracts the table from the doc comment:

```rust
/// The Lean file whose `#guard_msgs` block pins the model's outputs. Lean
/// checks that the block matches the model; this program checks that the
/// Rust implementation matches the block.
const LEAN_MODEL: &str = include_str!("../../lean/Book/Ch31/Differential.lean");

/// The lines of the expected-output table: from `info: ` to the end of the
/// doc comment.
fn expected_lines(source: &str) -> Vec<&str> {
    let start = source
        .find("info: sort")
        .expect("no table in the Lean file");
    let table = &source[start + "info: ".len()..];
```

```
    let end = table.find("-/").expect("unterminated table");
    table[..end]
        .lines()
        .map(str::trim)
        .filter(|l| !l.is_empty())
        .collect()
}

/// Parses a Lean list of integers such as `[-2, 0, 5]`.
fn parse_list(text: &str) -> Vec<i64> {
    let inner = text.trim().trim_start_matches('[').trim_end_matches(']');
    if inner.trim().is_empty() {
        return Vec::new();
    }
    inner
        .split(',')
        .map(|n| n.trim().parse().expect("an integer"))
        .collect()
}

/// Parses `some 3` or `none`.
fn parse_option(text: &str) -> Option<usize> {
    match text.trim() {
        "none" => None,
        other => Some(other.trim_start_matches("some ").parse().expect("an index")),
    }
}
```

Then it runs its own implementation on every input and compares:

```
/// Runs every line of the table through the Rust implementation.
/// Returns the number of sort and search cases checked.
fn differential_test() -> (usize, usize) {
    let (mut sorts, mut searches) = (0, 0);
    for line in expected_lines(LEAN_MODEL) {
        let (call, expected) = line.split_once(" = ").expect("`call = result`");
        if let Some(input) = call.strip_prefix("sort ") {
            let mut v = parse_list(input);
            insertion_sort(&mut v);
            assert_eq!(
                v,
                parse_list(expected),
                "differs from the Lean model: {line}"
            );
            sorts += 1;
        } else if let Some(args) = call.strip_prefix("search ") {
            let (list, key) = args.rsplit_once(' ').expect("`search list key`");
            let a = parse_list(list);
            let x: i64 = key.parse().expect("an integer key");
            let got = binary_search(&a, x);
            assert_eq!(
                got,
                parse_option(expected),
                "differs from the Lean model: {line}"
            );
            searches += 1;
        } else {
            panic!("unexpected line in the table: {line}");
        }
    }
    (sorts, searches)
}
```

There is a single source of truth. Lean checks that the table matches the model; Rust checks that the implementation matches the table. If the model changes, the Lean build fails until the table is updated, and then the Rust test runs against the new table without any copying. (Cargo notices files read by `include_str!` and recompiles the example when the Lean file changes.)

Step 4: property tests against the specification

A table of 24 cases is small. To cover more ground, the Rust program also checks the *specification* on thousands of random inputs, with a seeded generator written in a dozen lines (the workspace has no external dependencies):

```
/// SplitMix64: a tiny, seeded pseudo-random generator, so that the property
/// tests are reproducible.
struct SplitMix64(u64);

impl SplitMix64 {
    fn next(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }

    /// A number in `lo..=hi`.
    fn range(&mut self, lo: i64, hi: i64) -> i64 {
        lo + (self.next() % (hi - lo + 1) as u64) as i64
    }
}
```

```
/// Checks the two specifications on random inputs: the sort output is sorted
/// and a permutation of the input; the search result points at the key, or
/// the key is absent.
fn property_test(cases: usize) -> usize {
    let mut rng = SplitMix64(2026);
    let mut searches = 0;
    for _ in 0..cases {
        let len = rng.range(0, 20) as usize;
        let input: Vec<i64> = (0..len).map(|_| rng.range(-30, 30)).collect();

        let mut sorted = input.clone();
        insertion_sort(&mut sorted);
        assert!(
            sorted.windows(2).all(|w| w[0] <= w[1]),
            "not sorted: {sorted:?}"
        );
        // A permutation: the same multiset of elements. Sorting both sides
        // with the standard library's sort is one way to compare multisets.
        let mut reference = input.clone();
        reference.sort();
        assert_eq!(sorted, reference, "not a permutation of {input:?}");

        for x in -32..=32 {
            match binary_search(&sorted, x) {
                Some(i) => assert_eq!(sorted[i], x),
                None => assert!(!sorted.contains(&x)),
            }
            searches += 1;
        }
    }
    searches
}
```

The two checks complement each other. The differential test compares exact outputs with a model that is proved correct, on few inputs. The property test checks the two halves of the specification, sorted and permutation, found and absent, on many inputs, but with a hand-written checker that is itself unproved. (To check "is a permutation of the input" it compares with the standard library's sort; a different reference would do, as long as it is trustworthy.)

The result

```
let (sorts, searches) = differential_test();
println!("Lean model: {} expected results read", sorts + searches);
println!("  sort:   {sorts} cases agree with the Lean model");
println!("  search: {searches} cases agree with the Lean model");

let cases = 10_000;
let checked = property_test(cases);
println!("property tests: {cases} random lists sorted correctly");
println!("property tests: {checked} searches satisfy the specification");

// With duplicates, the specification of search does not fix the index.
let sevens = [7, 7, 7, 7];
let ours = binary_search(&sevens, 7);
let std_result = sevens.binary_search(&7);
println!("search {sevens:?} 7: ours {ours:?}, std {std_result:?}");
```

```
Lean model: 24 expected results read
  sort:   9 cases agree with the Lean model
  search: 15 cases agree with the Lean model
property tests: 10000 random lists sorted correctly
property tests: 650000 searches satisfy the specification
search [7, 7, 7, 7] 7: ours Some(2), std Ok(3)
```

The last line shows why the difference between "the specification determines the output" and "it does not" matters. On `[7, 7, 7, 7]`, our search returns index 2, and the standard library's `binary_search` returns index 3. Both are correct: the specification only requires *an* index holding 7, and the documentation of `slice::binary_search` says explicitly that any match may be returned when there are several. A differential test that compared our search with the standard library's would report a failure that is not a bug. Our table can compare exact indices only because the Lean model and the Rust code are the same algorithm. For sorting, no such care is needed: we proved that every correct sort returns the same list.

31.8 The landscape: verifying Rust itself

The workflow of this chapter keeps a gap open: the Lean model is proved, the Rust code is only tested against it. Several research and industrial tools aim to close that gap by verifying Rust code directly. They differ in where the specification lives, how much the user proves by hand, and what they guarantee. The descriptions below are brief and general; all of these tools are under active development, so check each project's documentation for its current capabilities and status.

Tool	Approach	Proofs written in
Aeneas	translates Rust into a pure functional model	Lean (and other provers)
Verus	specifications and proofs inside Rust syntax	Rust-like syntax, checked by an SMT solver
Creusot	deductive verification with contracts	annotations, discharged via the Why3 platform
Prusti	deductive verification with contracts	annotations, built on the Viper infrastructure
Kani	bounded model checking	test-like harnesses, no proofs

Aeneas is the closest to this chapter. It translates (a subset of) safe Rust into a purely functional program in Lean (other backends exist too), so that you prove theorems about the translation with

the techniques of this chapter. Its key idea is that, in safe Rust, the borrow checker's discipline makes it possible to model mutable borrows functionally: a function that takes `&mut T` becomes a function that returns the updated value. The model is generated rather than written by hand, which removes the gap between our `isort` and our `insertion_sort`; what remains to trust is the translation.

Verus lets you write specifications (`requires`, `ensures`, loop invariants) and proofs in a Rust-like syntax next to the code, with "ghost" code that exists only for verification, and discharges the proof obligations with an SMT solver, an automatic prover for logical formulas with arithmetic and data structures.

Creusot and **Prusti** are deductive verifiers in the same spirit: contracts written as attributes on Rust functions (preconditions, postconditions, invariants), translated to an intermediate verification platform (Why3 for Creusot, Viper for Prusti) whose automatic provers check them. Verifiers of this kind can also establish properties such as the absence of panics or of arithmetic overflow, which our Lean model could not see.

Kani is a bounded model checker. You write a harness that looks like a test but uses symbolic values (`kani::any()`) instead of concrete ones, and Kani explores *all* values, up to bounds such as the number of loop iterations, looking for panics, overflows, failed assertions and some kinds of undefined behavior in `unsafe` code. It proves nothing beyond its bounds, but within them it is exhaustive, and it needs no proofs from the user. Finally, **Flux** adds refinement types (the idea of Chapter 30's `Percent`, with checked predicates) to Rust.

What they share is the subject of this book: each tool exploits Rust's type system, and the ownership discipline in particular, to make verification tractable. Aliasing is the classic enemy of program proofs, and safe Rust rules out unrestricted aliased mutation (Part VI) before the verifier starts.

31.9 What a proof gives you, and what it does not

A Lean proof about a program is the strongest evidence of correctness this book offers. It is still worth being precise about what it covers.

The specification can be wrong or incomplete. A proof shows that the program meets the specification, not that the specification says what you meant. "Sorted" alone would have accepted `fun _ => []`. Our search specification does not fix which index is returned, which is a choice, not a bug, but it is the kind of thing you have to notice. And no part of our specification mentions cost: `isortCost_le` is a separate theorem, and without it a correct but exponential sort would satisfy everything else.

The model is not the implementation. Our proofs are about `isort` and `search`, Lean functions on lists and arrays of unbounded integers. The Rust code works in place, on `i64`, with `usize` indices. The differential and property tests connect the two on many inputs, not all of them. Tools like Aeneas narrow this gap by generating the model from the code.

Something is always trusted. The theorems rest on Lean's kernel and on three axioms; the `#eval` outputs rest on Lean's compiler; the Rust tests rest on rustc, the standard library, and our small parser

for the table; everything rests on the hardware. A proof does not remove trust; it moves trust from a large program you wrote to a small, carefully reviewed checker.

Rust's guarantees and Lean's are different in kind. Every Lean function terminates, and Lean's theorems hold for all inputs. A well-typed Rust function can loop or panic (Chapter 10). Proving a Rust program correct therefore includes properties, termination and the absence of panics, that a Lean model gets for free.

None of this makes proofs less valuable. It tells you where to look when something goes wrong, and it is the reason this chapter combined proofs with tests rather than choosing between them.

31.10 Common confusions

"**A verified program cannot fail.**" It satisfies its specification, under the assumptions of its trusted base. If the specification is incomplete, or the deployed code differs from the verified model, it can still fail.

"**The differential test proves the Rust code correct.**" It checks 24 inputs against a verified model. That is strong evidence about those inputs and some evidence about others; the property tests add many more inputs. Neither is a proof.

"**`partial def` is fine, because my function terminates in practice.**" It runs fine. But you can prove nothing about it, not even facts that `#eval` shows. If you will want theorems, give a measure or use fuel.

"**The induction hypothesis is whatever I need.**" It is the statement you are proving, for the smaller case. If the recursive call changes an accumulator or an interval, the statement must quantify over it (`generalizing`), or the hypothesis will not apply.

"**`simp` either works or it doesn't.**" `simp` rewrites with a large, changing set of lemmas, so a proof that works today can break when the library grows. Use `simp?` to see what it did and replace it with `simp only [...]` in proofs you want to keep stable.

"**`omega` and `linarith` are interchangeable.**" They overlap on linear arithmetic over `ℕ` and `ℤ`. `omega` also handles `/` and `%` by literals and reasons about truncated subtraction on `ℕ`; `linarith` works over ordered fields too and treats nonlinear subterms as opaque atoms. For polynomial *equations*, use `ring`.

31.11 Summary

- A specification is a type: a subtype `{ ys // ys.SortedLE ∧ ys ~ xs }` holds a result together with proofs of its properties. A good specification rules out every wrong answer; "sorted" and "permutation" are each insufficient, and together they determine the output uniquely.
- Induction must sometimes be generalized: when a recursive call changes an argument, prove the statement for all values of it, with `induction … generalizing`. Trees give one induction hypothesis per child.

- Lean requires termination. Structural recursion is automatic; well-founded recursion needs a decreasing measure (`termination_by`) and sometimes a hand-written proof (`decreasing_by`); lexicographic measures handle nested recursion; `partial` gives up reasoning, fuel restores totality. `fun_induction` proves properties of well-founded definitions.
- Insertion sort is verified with `List.Perm` and Mathlib's `List.SortedLE` ; it equals Mathlib's `insertionSort` and core Lean's `mergeSort` on every input, because sorted permutations are unique. Its cost is at most $n(n-1)/2$ comparisons.
- Binary search is verified on sorted arrays: a returned index holds the key, and `none` means the key is absent. The specification does not fix which index is returned among duplicates.
- The tactics: simp lemmas and `simp?` / `simp only` , `omega` for linear arithmetic with literals, `linarith` for linear reasoning with opaque atoms, `ring` for polynomial identities, `calc` for readable chains, `grind` for routine case analysis.
- Model in Lean, implement in Rust: the model's outputs, pinned by `#guard_msgs` , drive a differential test of the Rust code; property tests check the specification on random inputs.
- Aeneas, Verus, Creusot, Prusti and Kani verify Rust code directly, with different trade-offs. A proof is only as good as its specification and its trusted base.

31.12 Exercises

1. **Sorting twice changes nothing.** Prove `isort (isort xs) = isort xs` . *Hint:* do not use induction. `isort (isort xs)` is sorted and a permutation of `isort xs` ; `isort xs` is sorted and a permutation of itself. Use `sorted_perm_unique` ; the whole proof fits on one line.
2. **A merge, and its termination measure.** Define `merge : List ℤ → List ℤ → List ℤ` , which merges two sorted lists, with clauses for `[], ys` , `xs, []` and `x :: xs, y :: ys` . Lean accepts it without help. Use `termination_by?` to find out which measure it used, and explain why neither argument alone would do. Then prove `(merge xs ys).length = xs.length + ys.length` . *Hint:* Lean reports a lexicographic measure on the pair of arguments. For the length theorem, `fun_induction merge xs ys <;> simp_all <;> omega` .
3. **When the specification is not enough to compare outputs.** In `ch31_sort.rs` , change the midpoint to the *lower* middle, `lo + (hi - lo - 1) / 2` . Is the search still correct? Run the example: which checks fail, and why? What would you change in the Lean model, and in the table, to make the test pass again? *Hint:* the property tests still pass, because the search still meets its specification. The differential test fails on a duplicated key: for `-2` in `[-2, -2, 0, 5, 5]` the lower middle finds index 0, and the table says 1.
4. **The first occurrence.** Write a variant `lowerBound` of `searchFrom` that returns the *smallest* index `i` with `x ≤ a[i]` (or `a.size` if there is none), and prove, for sorted arrays, that every index below the result holds an element smaller than `x` . *Hint:* keep the invariant "everything below `lo` is `< x` , everything from `hi` on is `≥ x` ", and state the theorem for an arbitrary interval, as in `searchFrom_none` . The recursion has the same measure, with two branches instead of three (`a[mid] < x` goes right, everything else goes left), and returns `lo` when the interval is empty.

5. **A quadratic lower bound.** Prove that sorting a strictly decreasing list of length `n` costs exactly `n * (n - 1) / 2` comparisons, at least for `n ≤ 10` by `decide` on concrete lists, and then, if you are ambitious, for all `n`. *Hint:* for a strictly decreasing list, each insertion goes all the way to the end of the sorted tail, so `insertCost x (isort xs) = xs.length`. For concrete lists, `example : isortCost [5, 4, 3, 2, 1] = 10 := by decide`.
6. **Specifications on paper.** For each specification, find a function of type `List ℤ → List ℤ` that satisfies it but is not a sort: (a) "the output is sorted"; (b) "the output is a permutation of the input"; (c) "the output is sorted and has the same length as the input"; (d) "the output is sorted and every element of the output is an element of the input". *Hint:* for (c) and (d), think of lists with repeated elements, such as `[1, 2]` and `[1, 1]`.